01 — Upload & intake

What it does: accepts one PDF at a time into a case, validates it, stores the original immutably, records its hash, and fans out the automated pipeline. It detects nothing itself; it is the chain-of-custody starting point.

Code: `api/documents.py` (`upload_document`), `services/upload_validation.py` , `services/storage_service.py` , `services/usage_service.py` , `models/document.py` , frontend `pages/NewCasePage.tsx` + `components/upload/FileDropzone.tsx` + `lib/uploadLimits.ts` .

Flow

1. The UI creates the case (`POST /cases`), then uploads each selected file with its own `POST /cases/{case_id}/documents` (multipart field `file`) using `XMLHttpRequest` for progress. Files upload in parallel; a failed file can be retried without recreating the case, dismissed using the remove icon (`[X]`), or discarded completely via `Reset form` which cleanly restores the intake form and unlocks case type selection.
2. The endpoint 404s on an unknown case (or one of another company, or — for a `user` — one they did not submit; platform admins get 403), then **validates the file synchronously** (next section). A rejected file is never stored, never gets a `documents` row or audit event, never counts toward usage and never queues a task, so it costs no worker slot and no Azure call. A valid file's `sha256(content)` is computed.
3. It writes the bytes to Blob Storage at **`companies/{company_id}/cases/{case_id}/documents/{document_id}_{sha256}{original_extension}`** with `overwrite=False` . The path embeds the hash and the document's own id, so collisions are impossible even if a filename repeats. A storage failure is a **502**.
4. It inserts a `documents` row (`processing_status = pending` , `content_type = application/pdf`) and flushes, writes the audit event `document_uploaded` with `{original_filename, file_hash, file_size_bytes}` , bumps the company's usage counters (`documents_uploaded` , `files_stored` , `storage_bytes`) in the same transaction, and commits.
5. It enqueues six Celery tasks — each as `(document_id, company_id)` with a fair-share priority — and returns **201**:

Task	Queue	Module	Depends on	Notes
`process_document`	`extraction_queue`	`tasks/document_processing.py`	—	OCR + classify + extract; chains into `run_document_checks`
`run_metadata_forensics`	`forensics_queue`	`tasks/metadata_forensics_task.py`	file only	
`run_tampering_checks`	`forensics_queue`	`tasks/tampering_checks_task.py`	file only	ELA + copy-move share one render
`run_duplicate_check_task`	`forensics_queue`	`tasks/duplicate_check_task.py`	file only	compares within the company only
`run_visual_inconsistency_review_task`	`vision_queue`	`tasks/visual_inconsistency_task.py`	file only	also carries the AI-generation question
`run_signature_detection`	`vision_queue`	`tasks/signature_detection_task.py`	file only	

6. The response contains a fresh signed download URL (if signing fails it falls back to the durable URL; the upload itself has already succeeded).

Afterwards the UI offers an **optional** step to draw a reference signature (see 09).

Upload validation

`services/upload_validation.py`, called before anything is stored. **Only PDF documents are accepted** (decision of 2026-10-03: every forensic check is PDF-based, so an image would only ever get OCR and no tamper analysis). The checks run in this order and the first failure wins. Each rejection returns `detail: {"code", "message", ...}`, and the UI shows `message` as-is next to the file.

#	Check	HTTP	`code`	Message / extra fields
1	0 bytes	400	`file_empty`	"The file is empty (0 bytes)."
2	Over `MAX_UPLOAD_SIZE_BYTES` (default **10 MB** = 10,485,760 bytes)	413	`file_too_large`	States both sizes, e.g. "The file is 12.0 MB; the maximum allowed size is 10.0 MB."; `size_bytes`, `max_size_bytes`
3	Content is not a PDF, judged by the **header bytes** (`%PDF-`), never the name or declared MIME type	415	`unsupported_file_type`	"This file is a JPEG image. Only PDF files are accepted — please save or scan the document as a PDF." (the type is named when recognised: JPEG, PNG, TIFF, GIF, BMP images, ZIP/Office files)
4	A PDF whose name carries another type's extension (`.png`, `.docx`, …)	415	`file_type_mismatch`	`detected_type`, `extension`
5	The PDF needs a password to open	422	`file_password_protected`	"This file is password-protected. Please remove the password and re-upload."
6	The PDF can't be parsed locally (PyMuPDF)	422	`file_corrupted`	"The file appears to be corrupted or unreadable."

Details:

- **Magic bytes.** `%PDF-` anywhere in the first 1024 bytes (the PDF spec allows junk before the header; Acrobat and MuPDF accept it). An image renamed to `.pdf` is refused as an image.
- **Extensions.** Only a *recognised* file-type extension (images, Office, text, archives, …) is compared with the content. An unrecognised suffix (the ".2026" in "Invoice 12.03.2026") and a name with no extension are accepted on the content alone; `.PDF` in upper case is fine.
- **Parse.** PyMuPDF — the same library the pipeline renders pages with — opens the file and parses every page's content stream. MuPDF silently "repairs" a truncated file, so two cases count as corrupted: a file it repairs into **0 pages**, and a file it had to repair that has **no `%%EOF` marker in its last 1024 bytes** (a cut-off file can otherwise yield a few pages and silently lose the rest). A merely sloppy PDF with wrong cross-reference offsets still ends in `%%EOF` and is accepted. All 38 sample PDFs pass, including the 12 tampered samples (incremental saves).
- **Password policy: reject-and-ask** (🔶 decision, 2026-10-03). The server never asks for or applies a password, so the system never handles other people's document passwords. A PDF with only an **owner** password (print/copy restrictions; it opens without a password) is accepted. To revisit: the alternative is a password field on upload plus server-side decryption.
- **Stored `content_type`** is always `application/pdf` (the detected type), not the browser-declared one.
- The endpoint reads at most limit + 1 bytes into memory and parses in a worker thread, so the event loop isn't blocked. Uvicorn still receives (and spools to a temp file) the *whole* request body before the endpoint runs — cap the request size at the ingress / reverse proxy in production too (production deployment).

- **Frontend:** `lib/uploadLimits.ts` refuses files that are not named/typed as PDF, empty files and files over 10 MB before sending (the server check is the one that counts). The file picker only offers PDFs and the drop zone states the rules ("PDF only · up to 10.0 MB each · not password-protected"). Keep `MAX_UPLOAD_BYTES` in step with `MAX_UPLOAD_SIZE_BYTES`.
- Tests: `tests/test_upload_validation.py`.

Output shape

`DocumentResponse` (`schemas/document.py`):

```
{
  "id": "uuid", "case_id": "uuid",
  "original_filename": "Invoice.pdf",
  "content_type": "application/pdf",       // detected from the file's header bytes
  "file_size_bytes": 184223,
  "file_hash": "e3b0c4…",                  // SHA-256 hex, 64 chars
  "file_url": "https://…?sv=…&sig=…",      // short-lived SAS URL
  "uploaded_at": "2026-09-20T17:08:40Z",
  "processing_status": "pending"           // pending | processing | complete | failed
}
```

`documents.blob_storage_path` stores the **durable** (unsigned) blob URL. The container is private, so anything that needs the file mints a SAS URL with `StorageService.get_download_url()` (15 min default; 30 min for OCR); `ensure_company_blob()` refuses to sign a path under another company's prefix.

Invariants

- **Originals are write-once.** `StorageService` has `upload()` and `download_bytes()` but no overwrite/delete path. Every later stage only *reads* the original.
- **`file_hash` is the integrity anchor.** It is recomputed and compared when a report is generated (`HashVerification` in the report appendix).
- **Byte-identical re-uploads are not blocked.** `file_hash` is indexed but there is no uniqueness constraint; exact duplicates are caught by the *perceptual* check (10), not at upload.

Known limitations

- No page-count limit and no antivirus scan (type, size, corruption and password checks exist; see above).
- Any company user can create a case, but only its owner or a reviewer of the company can add to it.
- Documents stored before the PDF-only rule may be images; they were OCR'd but no forensic check runs on them, and `pipeline_status` only requires forensic checks for PDFs, so such a case can still be scored.
- One task failing to enqueue (Redis down) after the DB commit leaves a `pending` document that never progresses; there is no sweeper.
- If every file of a new case is rejected, the case itself (created first) remains, with no documents.

Risk rules fed

None directly. It produces the documents every rule evaluates.